

STUDENT 3

LLM Experiments, Metrics and Reproducibility

Run fair swarm-versus-centralized experiments and measure behavior, API use, time and cost.

ABMS-LLM Cyber Organizational Simulation

Beginner work plan for implementing LLM agents in the current repository. This document is written for both the assigned student and an AI coding assistant.

Main research question

How does agent behavior and performance differ when LLM-driven cyber agents are organized as an independent swarm versus a centralized hierarchy?

Important decision

- LLM cognition is now the main implementation target.
- The existing rule-based cognition stays small and working only as a baseline and API-failure fallback.
- Do not delete the rule baseline, because the research needs a control and developers need a cheap debugging mode.
- Work in the Conda environment named mesa and use Mesa 3.x APIs.

1. Your Job in Simple Words

You own the simulation loop, trial runner and results. You make sure both organizations are compared on the same starting scenarios and that every LLM call is measured. You do not write prompts or make UI charts.

How one LLM decision works

- 1 Student 1 turns the current simulation state into a small observation dictionary.
- 2 Student 2 sends that observation to the LLM with instructions and allowed actions.
- 3 The LLM returns one JSON decision. Student 2 checks it before trusting it.
- 4 Student 1's resolver applies the valid action and returns the result.
- 5 Student 3 records behavior, outcome, tokens, time, errors and estimated cost.
- 6 Student 4 shows safe decision summaries and metrics in the UI and documentation.

Words you need to know

Word	Simple meaning
LLM	A language model that reads an observation and chooses an action.
API	The online service used by Python to ask the LLM for a response.
Prompt	The instructions and simulation information sent to the LLM.
JSON	A strict text format used so Python can safely read the LLM answer.
Token	A small unit of text used to measure API usage and cost.
Fallback	A safe rule-based action used when the API or LLM answer fails.
Mock client	A fake API used in tests so tests are fast, free and repeatable.

2. Files You Own

File	What it should do
abms_cyber/model/cyber_model.py	Create components, run one step and stop the episode.
abms_cyber/metrics/data_collector.py	Collect per-step model and agent values.
abms_cyber/metrics/llm_metrics.py	Create counters for calls, tokens, latency, failures and fallback.
abms_cyber/metrics/performance.py	Create final episode measures.
abms_cyber/experiments/experiment_runner.py	Create repeatable multi-trial comparisons.
main.py and abms_cyber/run.py	Provide clear commands without placing experiment logic in entry files.
output/results/	Store tidy CSV results; never store API secrets.

What you must build

- A CyberModel step where centralized and swarm actions happen exactly once.
- Clear episode outcome: success plus target, detection or time-limit termination reason.
- Core metrics: success, steps, detection, compromised nodes, actions and duplicate actions.
- LLM metrics: call count, input/output/total tokens, response latency, API errors, invalid responses, retries, fallback count and estimated cost when pricing is configured.
- A trial runner that uses matched environment seeds for swarm and centralized runs.
- CSV outputs and a short console summary. Rule-based runs are only a baseline/control, not the main result.

Stay inside your role

Do not redesign another student's layer without agreement. If you need a new field or function, first write a small example of the input and output, then ask its owner to approve the interface.

3. Shared LLM Data Format

All students should use one simple data format. Student 1 builds the observation. Student 2 sends it to the LLM and validates the response. Students 3 and 4 read safe metadata from the result.

Example observation

```
{
  "agent_id": 1,
  "organization": "swarm",
  "current_node": "3",
  "privilege_level": 0,
  "known_nodes": ["3", "4"],
  "recent_actions": ["SCAN:3"],
  "detection_level": 0.20,
  "goal": "Compromise the target with admin privilege",
  "allowed_actions": ["SCAN", "MOVE", "EXPLOIT", "ESCALATE"]
}
```

Example LLM response

```
{
  "action": "EXPLOIT",
  "target_id": "3",
  "reason": "The current node is scanned and not compromised."
}
```

Safety rules

- Never execute Python code returned by the LLM.
- Never trust action or target text until it is checked against allowed values.
- Never hardcode, commit, print or display the API key.
- Use environment variables for secrets and fake clients for normal tests.
- Keep prompts small enough to control cost and store only safe logs.

4. Build It Step by Step

- 1 Agree one model-step sequence with Student 2 so no centralized worker acts twice.
- 2 Add explicit success and termination_reason fields to the model.
- 3 Create llm_metrics.py with a simple record_call method. Receive sanitized metadata from Student 2, not API secrets.
- 4 Extend DataCollector with organization, actions, LLM calls, tokens, latency, errors and fallback values.
- 5 Create experiment_runner.py. Each trial creates a fresh model and stores its seed, configuration and outcome.
- 6 Use the same seed list and the same LLM model/settings for both organizations. Organization should be the main intended difference.
- 7 Add an optional response cache or replay mode for debugging only. Label cached/replayed runs and do not mix them with live experimental results.
- 8 Export one row per trial, calculate mean/variance/success rate and report token/cost totals separately.
- 9 Start with 1-3 cheap smoke trials. Run large paid experiments only after tests and budget limits pass.

When using an AI coding assistant

- Give the assistant this PDF and the repository files named in your assignment.
- Ask it to inspect before editing and to explain what already exists.
- Request one small task at a time; review the diff before moving to the next task.
- Require tests with a mock API before making a paid live request.
- Do not accept code that hides errors, exposes secrets or changes unrelated files.

5. Tests and Definition of Done

Your work is done only when these checks pass:

- Each requested trial creates one independent model and one result row.
- Every termination condition records the correct reason.
- Known fake API usage produces the expected token, latency, retry and fallback totals.
- Matched seeds are used for the two organizations.
- CSV contains configuration, organization, seed, outcome and required LLM metrics.
- A maximum calls/tokens/cost limit can stop or skip unsafe experiment execution.
- Rule-based, mocked-LLM and live-LLM results are clearly labeled and never mixed silently.

Minimum evidence to submit

- List of files changed and one sentence explaining each file.
- Automated test output for normal, invalid and failure behavior.
- One end-to-end mocked-LLM run that costs nothing.
- One carefully limited live-LLM smoke run if the team has an API key and budget.
- A note listing remaining limitations and any assumptions agreed by the team.

Cost warning

Do not begin with many trials. Start with mocked responses, then one live step, then one short episode. Student 3 must add call/token/cost limits before the team runs large experiments.

6. Six-Week Plan and Team Handoffs

Week	Main result
1	Agree model-step and LLM usage event formats with Students 1 and 2.
2	Add outcome/termination state and model lifecycle tests.
3	Build LLM metric counters and DataCollector reporters.
4	Build matched-seed experiment runner and CSV export.
5	Run mocked and small live comparisons; validate budget controls.
6	Produce final tables, reproducibility notes and report-ready results.

Required team handoffs

- Student 1 gives Student 2 the observation fields and action-resolver interface.
- Student 2 gives Student 3 sanitized LLM usage events and gives Student 4 safe decision-log fields.
- Student 3 gives Student 4 stable DataCollector column names and termination status.
- Student 4 gives everyone the final runbook and reports integration defects to the correct owner.

Four contracts to agree in Week 1

- Observation JSON fields.
- LLM response/action JSON fields.
- One model-step sequence for swarm and centralized modes.
- LLM usage and decision-log fields for metrics and UI.

7. Prompt to Give an AI Coding Assistant

Attach or provide this PDF and then send the following prompt. The assistant must still inspect the real repository because this plan describes responsibilities, not the final code state.

You are helping Student 3 on AbstractCyberSim. Own CyberModel lifecycle, LLM usage metrics, performance metrics and repeatable experiments. Read Knowledge/Requierment, Knowledge/Important rules, model, metrics, main.py, and the interfaces from Students 1 and 2 before editing. Use Mesa 3.x. Ensure each action runs once, record success and termination reason, collect calls/tokens/latency/errors/retries/fallback/cost, and compare swarm versus centralized with matched seeds and equivalent LLM settings. Add budget limits and tests with fake usage data. Keep secrets out of results. Explain formulas simply, export tidy CSV data, and run tests plus small smoke trials.

Ask the AI to finish each reply with

- What it changed in beginner-friendly language.
- Which files it changed.
- Which tests and commands it ran and their results.
- Any unresolved problem or decision needed from another student.
- Whether any live API call was made and its approximate usage/cost.

Final reminder

The goal is not merely to make an API call. The goal is to create a controlled research system where valid LLM decisions can be compared fairly between swarm and centralized organizations.